

Занятие 1: Введение в серверную разработку

Архитектура веб-приложений, обзор экосистемы

Слайд 1: Клиент-серверная архитектура

Клиент (Frontend)

- Браузер или мобильное приложение
- Отправляет HTTP-запросы
- Отображает пользовательский интерфейс

Сервер (Backend)

- Обработывает запросы
- Выполняет бизнес-логику
- Взаимодействует с БД и внешними сервисами
- Возвращает HTTP-ответы

```
[Client] <--HTTP--> [Server] <--SQL--> [Database]
                    |
                    +--> [External APIs]
```

Слайд 2: Архитектурные паттерны

MVC (Model-View-Controller)

- **Model:** Данные и бизнес-логика
- **View:** Представление данных
- **Controller:** Обработка запросов, связывание Model и View

MVT (Model-View-Template) - Django

- **Model:** Данные
- **View:** Контроллер (обработка запросов)
- **Template:** Представление

Layered Architecture

- Presentation Layer
- Business Logic Layer
- Data Access Layer
- Database Layer

Слайд 3: Сравнение фреймворков

Характеристика	Django	FastAPI	Express (Node.js)
Язык	Python	Python	JavaScript
Архитектура	MVT	Асинхронная	Middleware
ORM	Встроенный	SQLAlchemy/SQLModel	Mongoose/Sequelize
Скорость	Средняя	Высокая	Высокая
Learning curve	Средний	Низкий	Низкий
Для чего	Полнофункциональные приложения	API, микросервисы	API, реалтайм

Слайд 4: Монолит vs Микросервисы

Монолит

- Одно приложение с всем функционалом
- Простое развертывание
- Сложности при масштабировании
- Тесная связанность кода

Микросервисы

- Независимые сервисы
- Каждый сервис - отдельное приложение
- Масштабируется по отдельности
- Сложная инфраструктура
- Требуется оркестрация (Kubernetes, Docker Compose)

Слайд 5: Экосистема Python для backend

Фреймворки:

- Django - "batteries included"
- FastAPI - современный, быстрый
- Flask - минималистичный

Инструменты:

- Virtualenv / Poetry - управление зависимостями
- Gunicorn / Uvicorn - WSGI/ASGI серверы
- Celery - фоновые задачи
- Redis - кэширование, очереди

Слайд 6: Экосистема Node.js для backend

Фреймворки:

- Express - классический
- NestJS - архитектурный (TypeScript)
- Koa - современный

Инструменты:

- npm / yarn - пакетные менеджеры
- PM2 - управление процессами
- MongoDB - NoSQL база данных
- Socket.io - WebSocket

Слайд 7: Структура проекта Django

```
myproject/
├── manage.py
├── myproject/
│   ├── __init__.py
│   ├── settings.py
│   ├── urls.py
│   └── wsgi.py
└── myapp/
    ├── __init__.py
    ├── models.py
    ├── views.py
    ├── urls.py
    └── templates/
```

Слайд 8: Структура проекта FastAPI

```
myproject/  
├── main.py  
├── requirements.txt  
├── app/  
│   ├── __init__.py  
│   ├── main.py  
│   ├── models/  
│   ├── routers/  
│   └── database.py  
└── tests/
```

Слайд 9: Практическая часть

Шаг 1: Создание виртуального окружения

```
python -m venv .venv  
source .venv/bin/activate # Linux/Mac  
.venv\Scripts\activate   # Windows
```

Шаг 2: Установка фреймворка

```
pip install django  
# или  
pip install fastapi uvicorn
```

Шаг 3: Создание проекта

```
django-admin startproject myproject  
# или  
# Создать main.py для FastAPI
```

Слайд 10: Hello World - Django

```
# views.py
from django.http import HttpResponse

def hello(request):
    return HttpResponse("Hello, World!")

# urls.py
from django.urls import path
from . import views

urlpatterns = [
    path('hello/', views.hello),
]
```

Слайд 11: Hello World - FastAPI

```
# main.py
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def read_root():
    return {"message": "Hello, World"}

@app.get("/hello/{name}")
def read_item(name: str):
    return {"message": f"Hello, {name}"}
```

Запуск:

```
uvicorn main:app --reload
```

Слайд 12: Домашнее задание

1. Установить Python и создать виртуальное окружение
2. Установить Django или FastAPI
3. Создать базовый проект
4. Реализовать 3 endpoint:
 - `/` - приветствие
 - `/about` - информация о студенте
 - `/time` - текущее время
5. Изучить документацию выбранного фреймворка

Слайд 13: Полезные ресурсы

Документация:

- Django: <https://docs.djangoproject.com/>
- FastAPI: <https://fastapi.tiangolo.com/>
- Express: <https://expressjs.com/>

Книги:

- "Two Scoops of Django" - Daniel Roy Greenfeld
- "Flask Web Development" - Miguel Grinberg

Курсы:

- Django Girls Tutorial
- FastAPI Official Tutorial